Basin MCMC: Discussion

Stella Srzich (3371930)

August 17, 2026

This document is meant to serve as a discussion of the Basin MCMC algorithm.

1 What kind of algorithm do we want?

In general, we want an algorithm that correctly samples from a target density $\pi(\cdot)$ (i.e., produces a Markov chain that is ergodic with respect to $\pi(\cdot)$). Moreover, we would like the chain to converge to the target density within a reasonable number of iterations. Moreover, out of all such algorithms, we would like to minimize the computational cost per effective sample.

The standard algorithm that indeed correctly samples from a target density $\pi(\cdot)$ is the Metropolis–Hastings (MH) algorithm.

Algorithm 1 Metropolis–Hastings (MH)

function: $[x^1, \ldots, x^N] = \mathrm{MH}\big(\pi(\cdot), q(\cdot \mid \cdot), x^0, N\big)$
input: target density $\pi(\cdot)$, proposal density $q(\cdot \mid \cdot)$, initial state x^0, number of steps N
output: ordered list of states $[x^1, \ldots, x^N]$

1: **for** $j = 0$ to $N - 1$ **do**
2: Given x^j, generate a proposal y distributed as $q(y \mid x^j)$.
3: Accept proposal y as the next state, i.e. set $x^{j+1} = y$, with probability

$$\alpha(y \mid x^j) = \min\left\{1, \frac{\pi(y)q(x^j \mid y)}{\pi(x^j)q(y \mid x^j)}\right\}.$$

4: Otherwise, reject y and set $x^{j+1} = x^j$.
5: **end for**

However, when the target density $\pi(\cdot)$ is multimodal (e.g., as is the case when we use high-level representations for states), with basic choices of proposal kernel $q(\cdot \mid \cdot)$, the MH algorithm can struggle to explore all modes within a reasonable number of iterations. In this case, the MH algorithm will not converge to the target density within a reasonable number of iterations.

One way to address this is to represent the target density as a collection of local probability distributions, which we call *basins*. Each basin has a representative state and describes states that are similar to that representative state. The idea is that each basin captures one local region, or mode, of the target density.

Instead of directly moving between individual states using only basic proposals (e.g., birth, death, and perturbation moves), we treat the state of the algorithm as a collection of basins. We then perform moves that add, remove, perturb, merge, or split these basins. The perturb move runs a local sampler within a basin that explores

nearby states and adapts the basin's probability distribution and probability so that it better represents the local shape of the target density.

I loosely view this algorithm as fitting the proposal kernel to the target density, including all its modes. Local moves allow the algorithm to fit a discovered mode, while birth, death, merge, and split moves allow it to discover new modes and change its representation of the existing modes. Moreover, since this algorithm represents the proposal kernel as a collection of local probability distributions (that we shall choose to be unimodal and easy to sample from), we can sample independently and efficiently from the proposal kernel.

The hope is that this allows the algorithm to explore a multimodal target density more effectively than directly applying basic proposal kernels to individual states. That is, to allow the chain to converge to the multimodal target density within a reasonable number of iterations. And, since the proposal kernel will be close to the target density and give samples that are effectively independent of each other, samples are effectively independent, and so the computational cost per effective sample is reduced.

Once this is achieved, one can worry about tweaking the algorithm to further reduce the computational cost per effective sample by decreasing the cost per sample (e.g., by using a delayed acceptance step to reduce the number of times we need to evaluate the forward map).

This idea is implemented in the Basin MCMC algorithm.

2 The Basin MCMC algorithm

2.1 Basins

We first discuss the notion of a basin. One way to think of a basin is as a probability distribution over states that captures how *similar* states are to some representative state. In particular, states are considered to be similar to the representative state (e.g., they have high probability density within its basin) if they (are similar in spatial location and) produce similar data (say, if we treat the data generated by the representative state as true data, then the posterior probability density of the state given the data is high).

To be more precise, a basin B is a set of the following:

- A **representative state** x_0.
- A **probability distribution** $\pi_B(\cdot)$ over states. We can think of this as consisting of:
  - A **sequence of probability distributions** $\{\pi_{B,n}(\cdot)\}_{n\in\mathbb{N}}$ each over the set of states with n atoms $\mathcal{X}_n$. Note that in cases where the order of states does not matter (as it does not in our GPR application), $\pi_{B,n}(\cdot)$ should be exchangeable. Thus, for each n, every atom has the same marginal distribution, and $\pi_{B,n}(\cdot)$ can be characterised by a distribution over the components of an individual atom together with a dependence structure between the n atoms.
  - A **sequence of probabilities** $\{p_{B,n}\}_{n\in\mathbb{N}}$ each defining the probability of a

state having n atoms, given the state is in basin B (e.g., it is distributed by $\pi_B(\cdot)$).

- A **probability** p_B that the state is in basin B (e.g., it is distributed by $\pi_B(\cdot)$).
- (Maybe?) a **local sampler** that generates samples from the posterior, proposed by $\pi_B(\cdot)$, and adapts $\pi_B(\cdot)$ and p_B with each iteration.

2.2 Moves on basins

Recall that our original MCMC algorithm worked by doing a mixture of moves between states of the model, such as birth, death, and perturbation moves. The basin MCMC algorithm will work in a similar way, still doing some kind of mixture of moves, but instead we will consider the state of the MCMC to be the set of basins (which implies a state of the model by unioning the representative states of the basins), and we will do moves on these basins.

The moves we will do on basins are:

- **Birth move:** add a new basin to the state.
- **Death move:** remove a basin from the state.
- **Perturb move:** run the local sampler within the basin, adapting the probability distribution and probability of the basin, and update the representative state of the basin to be the final state of the local sampler.
- **Merge move:** merge two basins into one, where the representative state of the new basin is the union of the representative states of the two basins.
- **Split move:** split a basin into two, where the representative state of the new basins is a partition of the representative state of the original basin.

To ensure the mixture of moves is in detailed balance with the target density it suffices to ensure each of the moves is in detailed balance with the target density. To achieve this, we must combine the birth and death moves, and the merge and split moves, into a single move. We then ensure each of the birth/death, perturb, and merge/split moves is in detailed balance with the target density with an MH step.

2.3 Algorithm

The algorithm roughly works as follows:

Algorithm 2 Basin MCMC

function: $[x^1, \ldots, x^N] = \text{BasinMCMC}\left(\pi(\cdot), q_{Birth/Death}(\cdot \mid \cdot), q_{Perturb}(\cdot \mid \cdot), q_{Merge/Split}(\cdot \mid \cdot), x^0, \pi_B(\cdot)\right.$
input: target density $\pi(\cdot)$, birth/death proposal kernel $q_{Birth/Death}(\cdot \mid \cdot)$, perturb proposal kernel $q_{Perturb}(\cdot \mid \cdot)$, merge/split proposal kernel $q_{Merge/Split}(\cdot \mid \cdot)$, initial state x^0, initial basin probability distribution $\pi_B(\cdot)$ (dependent on representative state x^0), number of steps N
output: ordered list of states $[x^1, \ldots, x^N]$

Initialize list of basins $\mathcal{B}^0 = [B]$ with $B = (x^0, \pi_B(\cdot), p_B = 1)$.
for $j = 0$ to $N - 1$ **do**
    Given $\mathcal{B}^j$, choose a move from $\{Birth/Death, Perturb, Merge/Split\}$.
    if Birth/Death: **then**
        Propose a list of basins $\mathcal{B}^*$ from $q_{Birth/Death}(\cdot \mid \mathcal{B}^j)$, where $\mathcal{B}^*$ is $\mathcal{B}^j$ with an additional or removed basin.
        Compute MH acceptance probability $\alpha(\mathcal{B}^* \mid \mathcal{B}^j)$ for Birth/Death move.
    else if Perturb: **then**
        Propose a list of basins $\mathcal{B}^*$ from $q_{Perturb}(\cdot \mid \mathcal{B}^j)$, where $\mathcal{B}^*$ is $\mathcal{B}^j$ with some basin's probability distribution and probability adapted, and representative state updated. This is done by running the local sampler of some basin.
        Compute MH acceptance probability $\alpha(\mathcal{B}^* \mid \mathcal{B}^j)$ for Perturb move.
    else if Merge/Split: **then**
        Propose a list of basins $\mathcal{B}^*$ from $q_{Merge/Split}(\cdot \mid \mathcal{B}^j)$, where $\mathcal{B}^*$ is $\mathcal{B}^j$ with two merged or split basins.
        Compute MH acceptance probability $\alpha(\mathcal{B}^* \mid \mathcal{B}^j)$ for Merge/Split move.
    end if
    Accept proposal $\mathcal{B}^*$, i.e. set $\mathcal{B}^{j+1} = \mathcal{B}^*$, with probability $\alpha(\mathcal{B}^* \mid \mathcal{B}^j)$.
    Otherwise, reject $\mathcal{B}^*$ and set $\mathcal{B}^{j+1} = \mathcal{B}^j$.
    Set $x^{j+1} = \bigcup_{B \in \mathcal{B}^{j+1}} B$.
end for

2.4 Development

This can become a fairly sophisticated algorithm. To develop it, I will complete the following tasks, implementing them in Python as I go:

1. **Decide initial basin probability distribution and probability:** Decide how to choose the initial probability distribution and probability for a basin given an initial representative state.
2. **Decide birth/death move:** Decide what the birth/death move should be.
3. **Decide perturb move:** Decide what the perturb moves should be. That is, decide what the local sampler for each basin should be. This includes how we propose from the basin probability distribution, and how we adapt the basin probability distribution and probability.
4. **Decide merge/split move:** Decide what the merge/split move should be.
5. **Decide move choice:** Decide how to choose a move from the set of possible moves.
6. **Prove diminishing adaption:** Prove that the adaption is diminishing, and so the chain is ergodic. Note that by construction, this MCMC is ergodic without

any adaption (as long as the birth move supports the target density, etc).

If I have extra time, I will also complete the following tasks:

1. **Include adaption stopping mechanism:** Decide a mechanism for stopping adaption of basins once they have diminishing adaption.
2. **Include delayed acceptance step:** Decide a delayed acceptance step before computing the MH acceptance probability and doing the MH step. We would use a bogus likelihood for the delayed acceptance step, and then use the actual likelihood for the MH step. This would reduce the number of times we need to evaluate the forward map and should increase efficiency with a well-chosen bogus likelihood.
3. **Consider adaption of basin probabilities:** Consider adding a step after acceptance/rejection that given $\alpha(\mathcal{B}^* \mid \mathcal{B}^j)$, adapts the probabilities of the basins in $\mathcal{B}^{j+1}$. I will probably not do this as it complicates the algorithm. I am only considering this as one may want to use all evaluations of the forward map (as is done to evaluate $\alpha(\mathcal{B}^* \mid \mathcal{B}^j)$) to adapt the basins.

2.5 Initial basin probability distribution and probability

2.5.1 Probability distribution ($n = 1$)

Suppose we have a representative atom $\theta_0 = (x_0, d, r, a)^\mathsf{T}$. We want to approximate the log posterior on $\mathcal{X}_1$, where the observed data is the noise-free data $y = A(\theta_0)$ generated by this atom.

To do this, we will do a second-order Taylor expansion about the representative atom θ_0 of the log posterior, given by

$$\log p(\theta \mid y) = -\frac{1}{2\sigma^2}\|y - A(\theta)\|^2 + \log p(\theta)$$

(up to an additive constant).

Writing

$$\delta\theta = \theta - \theta_0,$$

the second-order expansion is

$$\log p(\theta \mid y) \approx \log p(\theta_0 \mid y) + g^\mathsf{T}\delta\theta - \frac{1}{2}\delta\theta^\mathsf{T} P \delta\theta,$$

where

$$g = \nabla \log p(\theta_0 \mid y)$$

is the gradient of the log posterior and

$$P = -\nabla^2 \log p(\theta_0 \mid y)$$

is the negative Hessian, or precision matrix.

Gradient of the log posterior

The gradient of the log posterior is given by

$$g = \nabla \log p(\theta_0 \mid y) = \nabla \log p(y \mid \theta_0) + \nabla \log p(\theta_0).$$

The gradient of the log likelihood is

$$\nabla \log p(y \mid \theta) = \frac{1}{\sigma^2} J(\theta)^\mathsf{T} e(\theta),$$

where $e(\theta) = y - A(\theta)$ and $J(\theta) = \frac{\partial A(\theta)}{\partial \theta}$ is the Jacobian of the forward map. Since $y = A(\theta_0)$ gives $e(\theta_0) = 0$, we have that

$$\nabla \log p(y \mid \theta_0) = 0.$$

Thus, the gradient of the log posterior is simply the gradient of the log prior,

$$g = \nabla \log p(\theta_0).$$

Under the current prior, x_0, d, and a are uniform on their allowed ranges, while

$$p(r) = \lambda_r e^{-\lambda_r r}, \qquad r \geq 0.$$

Hence,

$$g = \begin{pmatrix} 0 \\ 0 \\ -\lambda_r \\ 0 \end{pmatrix}.$$

Hessian of the log posterior

The negative Hessian of the log posterior is given by

$$P = -\nabla^2 \log p(\theta_0 \mid y) = -\nabla^2 \log p(y \mid \theta_0) - \nabla^2 \log p(\theta_0).$$

The uniform priors have zero Hessian in the interior of their supports, and the log of the exponential radius prior is linear in r, and so

$$\nabla^2 \log p(\theta_0) = 0.$$

Thus, the negative Hessian of the log posterior is simply the negative Hessian of the log likelihood,

$$P = -\nabla^2 \log p(y \mid \theta_0).$$

Differentiating the log likelihood gradient gives

$$\nabla^2 \log p(y \mid \theta) = -\frac{1}{\sigma^2} J(\theta)^\mathsf{T} J(\theta) + \frac{1}{\sigma^2} \sum_k e_k(\theta) \nabla^2 A_k(\theta).$$

At $\theta = \theta_0$, recalling that $e(\theta_0) = 0$, we have

$$\nabla^2 \log p(y \mid \theta_0) = -\frac{1}{\sigma^2} J(\theta_0)^\mathsf{T} J(\theta_0).$$

It follows that the negative Hessian of the log posterior is

$$P = \frac{1}{\sigma^2} J(\theta_0)^\mathsf{T} J(\theta_0).$$

If P is positive definite, completing the square gives

$$\log p(\theta \mid y) \approx C - \frac{1}{2}(\theta - \mu)^\mathsf{T} P(\theta - \mu),$$

where

$$\mu = \theta_0 + P^{-1} g, \qquad \Sigma = P^{-1}.$$

Thus, after restricting it to the valid parameter domain, the local basin approximation is a truncated Gaussian with mean parameter μ and covariance parameter Σ.

In the implementation, the Moore–Penrose pseudoinverse P^+ is used in place of P^{-1} for numerical robustness when P is ill-conditioned:

$$\mu = \theta_0 + P^{+} g, \qquad \Sigma = P^{+}.$$

If all eigenvalues of P are positive, this agrees with P^{-1}, up to the numerical tolerance used in computing the pseudoinverse. Small eigenvalues correspond to weakly identified directions and therefore large local posterior variances.

Analytic forward-map Jacobian

To compute P, we require the Jacobian of the forward map. The forward map at (t, x) with parameters $\theta = (x_0, d, R, a)^\mathsf{T}$ is given by

$$A(t, x; \theta) = aB(x)w(t - T(x))$$

where

$$B(x) = \sqrt{\frac{d+R}{s(x)}} e^{-2\alpha s(x)}, \quad T(x) = \frac{2}{v}(s(x) - (d+R)) + \frac{2d}{v} = \frac{2}{v}(s(x) - R)$$

with $s(x) = \sqrt{(d+R)^2 + (x - x_0)^2}$, and

$$w(\tau) = (1 - 2(\pi f_0 \tau)^2) e^{-(\pi f_0 \tau)^2}.$$

We have that

$$\begin{aligned}
w'(\tau) &= 2(\pi f_0)^2 \tau (2(\pi f_0 \tau)^2 - 3) e^{-(\pi f_0 \tau)^2}, \\
\frac{\partial \log B(x)}{\partial x_0} &= (x - x_0) \left(\frac{1}{2s(x)^2} + \frac{2\alpha}{s(x)} \right), \\
\frac{\partial \log B}{\partial (d+R)} &= \frac{1}{2(d+R)} - \frac{(d+R)}{2s(x)^2} - \frac{2\alpha(d+R)}{s(x)}, \\
-\frac{\partial T(x)}{\partial x_0} &= \frac{2(x - x_0)}{v s(x)}, \\
-\frac{\partial T(x)}{\partial d} &= -\frac{2(d+R)}{v s(x)},
\end{aligned}$$

$$-\frac{\partial T(x)}{\partial R} = \frac{2}{v}\left(1 - \frac{(d+R)}{s(x)}\right).$$

Applying the product and chain rules gives

$$\begin{aligned}
\frac{\partial A(t,x;\theta)}{\partial x_0} &= aB\left[w(t-T(x))\frac{\partial \log B}{\partial x_0} + w'(t-T(x))\frac{-\partial T(x)}{\partial x_0}\right],\\
\frac{\partial A(t,x;\theta)}{\partial d} &= aB\left[w(t-T(x))\frac{\partial \log B}{\partial (d+R)} + w'(t-T(x))\frac{-\partial T(x)}{\partial d}\right],\\
\frac{\partial A(t,x;\theta)}{\partial R} &= aB\left[w(t-T(x))\frac{\partial \log B}{\partial (d+R)} + w'(t-T(x))\frac{-\partial T(x)}{\partial R}\right],\\
\frac{\partial A(t,x;\theta)}{\partial a} &= B\,w(t-T(x)).
\end{aligned}$$

Vectorising these four derivative images gives the four columns of J_0. The local precision, covariance, and recentered mean then follow from

$$P = \frac{1}{\sigma^2}J_0^{\mathsf{T}}J_0, \qquad \Sigma = P^{+}, \qquad \mu = \theta_0 + \Sigma g.$$

2.5.2 Probability distribution ($n = 2$)

Suppose we have a representative atom $\theta_0 = (x_{00}, d_0, R_0, a_0)^{\mathsf{T}}$. We want to approximate the log posterior on $\mathcal{X}_2$, where the observed data is the noise-free data $y = A(\theta_0)$ generated by this atom.

For simplicity, we will start by approximating the log likelihood on $\mathcal{X}_2$, where the observed data is the noise-free data $y = A(\theta_0)$ generated by this atom.

This is given by

$$\log p(y \mid \theta_1, \theta_2) = -\frac{1}{2\sigma^2}\left\|A(\theta_0) - A(\theta_1) - A(\theta_2)\right\|_2^2 + C$$

for some constant C. Recall that the forward map is given by

$$A(t,x;\theta) = aB(x)w(t-T(x))$$

where

$$B(x) = \sqrt{\frac{d+R}{s(x)}}e^{-2\alpha s(x)}, \quad T(x) = \frac{2}{v}(s(x) - (d+R)) + \frac{2d}{v} = \frac{2}{v}(s(x) - R)$$

with $s(x) = \sqrt{(d+R)^2 + (x-x_0)^2}$, and

$$w(\tau) = (1 - 2(\pi f_0 \tau)^2)e^{-(\pi f_0 \tau)^2}.$$

2.5.3 Reparameterization

Define $g_0 := (x_0, d_0, R_0)^{\mathsf{T}}$, so that we may write the representative atom as

$$\theta_0 = (g_0, a_0)^{\mathsf{T}}.$$

Now, with two atoms

$$\theta_1 = (g_1, a_1)^{\mathsf{T}} \quad \text{and} \quad \theta_2 = (g_2, a_2)^{\mathsf{T}},$$

where $g_1 := (x_1, d_1, R_1)^{\mathsf{T}}$ and $g_2 := (x_2, d_2, R_2)^{\mathsf{T}}$, we will use the following reparameterization:

- $m := a_1 + a_2$ is the total amplitude,
- $s := \frac{a_1}{a_1+a_2}$ is the amplitude split,
- $c := sg_1 + (1-s)g_2$ is the amplitude-weighted center,
- $z := \sqrt{s(1-s)}\,(g_1 - g_2)$ is the amplitude-scaled difference.

We may recover the original parameters as follows:

- $a_1 = sm$,
- $a_2 = (1-s)m$,
- $g_1 = c + \sqrt{\frac{1-s}{s}}z$,
- $g_2 = c - \sqrt{\frac{s}{1-s}}z$.